

RESOLUÇÃO PRÁTICA – KOTLIN

Nome: Luiz Philipe Alves Anselmo

Curso: Engenharia da Computação

Matrícula: 03360785

Turno: Noturno

Professor: Emerson Domingos

Disciplina: Desenvolvimento Mobile

Questão 1: Sistema de Cupons Avançado

when & Null Safety

Aplica descontos com base em um cupom promocional, tratando valores nulos ou cupons desconhecidos.

```
fun calcularDesconto(valor: Double, cupom: String?): Double {
    return when (cupom) {
        "PROMO10" -> valor - 10.0
        "PROMO20" -> valor - 20.0
        else -> valor
    }
}

fun main() {
    val preco = 100.0
    println("Com PROMO10: R$ ${calcularDesconto(preco, "PROMO10")}")
    println("Com PROMO20: R$ ${calcularDesconto(preco, "PROMO20")}")
    println("Sem cupom (null): R$ ${calcularDesconto(preco, null)}")
}
```

SAÍDA NO CONSOLE

Com PROMO10: R\$ 90.0

Com PROMO20: R\$ 80.0

Sem cupom (null): R\$ 100.0

Questão 2: Auditoria de Entregas

for & Elvis Operator

Percorre a lista de endereços substituindo nulos pelo valor padrão usando o Operador Elvis (? :).

```
fun auditarEntregas(enderecos: List<String?>) {
    for (endereço in enderecos) {
        val local = endereço ?: "Endereço Desconhecido"
        if (local == "Endereço Desconhecido") {
            println("Entrega Pendente: Falta de dados")
        } else {
            println("Rota traçada para: $local")
        }
    }
}

fun main() {
    val lista = listOf("Av. Paulista, 1000", null, "Rua das Flores, 123")
    auditarEntregas(lista)
}
```

SAÍDA NO CONSOLE

Rota traçada para: Av. Paulista, 1000

Entrega Pendente: Falta de dados

Rota traçada para: Rua das Flores, 123

Questão 3: Validação de Perfil de Streaming

Safe Call & Elvis Operator

Verifica o tamanho da biografia usando a Chamada Segura (?.) e o Operador Elvis (?:).

```
fun validarBioInfantil(bio: String?) {
    val tamanho = bio?.length ?: 0
    if (tamanho <= 50) {
        println("Bio aceita")
    } else {
        println("Bio muito longa")
    }
}

fun main() {
    validarBioInfantil("Gosto de desenhar e jogar futebol!")
    validarBioInfantil("Texto longo com mais de 50 caracteres para testar a validação.")
    validarBioInfantil(null)
}
```

SAÍDA NO CONSOLE

Bio aceita | Bio muito longa | Bio aceita

Questão 4: Processamento de Transações Pix

Laço & if/else

Soma os valores não nulos das transferências e descarta transações com erro.

```
fun main() {
    val transacoes: List<Double?> = listOf(50.0, null, 120.5, null, 10.0)
    var total = 0.0

    for (valor in transacoes) {
        if (valor != null) {
            total += valor
        } else {
            println("Transação ignorada")
        }
    }
    println("Total processado: R$ $total")
}
```

SAÍDA NO CONSOLE

Transação ignorada | Transação ignorada | Total processado: R\$ 180.5

Questão 5: Classificação de Feedback de Motoristas

when & Null Safety

Converte avaliações nulas para 0 com o Operador Elvis e classifica o feedback usando when.

```
// Resolução Atualizada da Questão 5
fun avaliarMotorista(nota: Int?) {
    val notaValida = nota ?: 0
    when (notaValida) {
        5 -> println("Excelente corrida!")
        4 -> println("Boa corrida.")
        1, 2, 3 -> println("Precisamos melhorar.")
        0 -> println("Nenhuma avaliação fornecida.")
        else -> println("Nota inválida.")
    }
}

fun main() {
    avaliarMotorista(5)
    avaliarMotorista(4)
}
```

Questão 6: Função Lambda para Cálculo de Gorjeta

Lambda & Implicit Parameter 'it'

Utiliza expressão Lambda e parâmetro implícito `it` para validar gorjeta opcional.

```
val calcularGorjeta: (Double?) -> Double = {
    if (it == null || it < 0.0) {
        0.0
    } else {
        it
    }
}

fun main() {
    println("Gorjeta R$ 15.0: R$ ${calcularGorjeta(15.0)}")
    println("Gorjeta nula: R$ ${calcularGorjeta(null)}")
    println("Gorjeta negativa (-5.0): R$ ${calcularGorjeta(-5.0)}")
}
```

SAÍDA NO CONSOLE

Gorjeta R\$ 15.0: R\$ 15.0 | Gorjeta nula: R\$ 0.0 | Gorjeta negativa (-5.0): R\$ 0.0

Questão 7: Limpeza de Banco de Dados de Usuários

Loops & Múltiplas Condições

Contabiliza e-mails nulos ou vazios usando Chamada Segura e Operador Elvis em laço de repetição.

```
fun limparBancoDados(emails: List<String?>) {
    var contasInvalidas = 0

    for (email in emails) {
        val tamanho = email?.length ?: 0
        if (email == null || tamanho == 0) {
            contasInvalidas++
            println("Aviso: Conta inválida detectada ($email). Marcada para deleção.")
        } else {
            println("Conta válida: $email")
        }
    }
    println("Total de contas que precisam ser apagadas: $contasInvalidas")
}

fun main() {
    val emails = listOf("usuario@email.com", null, "")
    limparBancoDados(emails)
}
```

SAÍDA NO CONSOLE

Conta válida: usuario@email.com
Aviso: Conta inválida detectada (null). Marcada para deleção.
Aviso: Conta inválida detectada (). Marcada para deleção.
Total de contas que precisam ser apagadas: 2